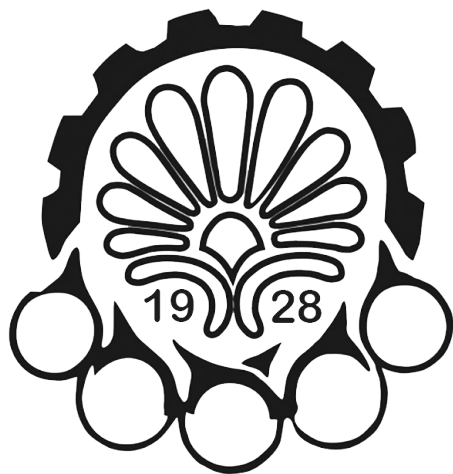




تمرین 5

یادگیری ماشین

آذر 1403

حسین کاوند 402112069

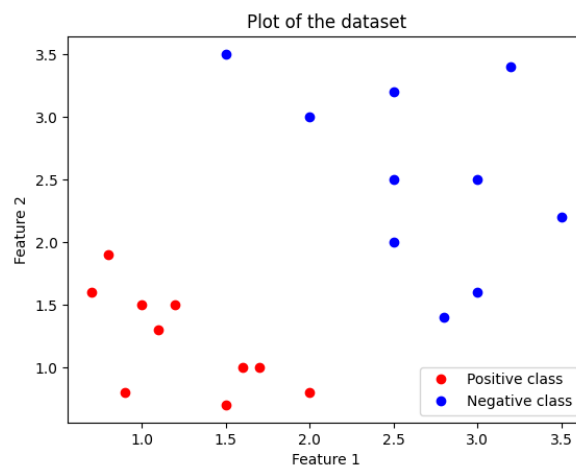
1. سوالات تشریحی

برای مجموعه داده اول، از آنجا که تفاوت داده های دسته های اول و دوم در فاصله آن ها با نقطه صفر است، می توان یک فیچر اضافه کرد که در آن فاصله هر داده با نقطه صفر $(x^2 + y^2)$ قرار دارد.

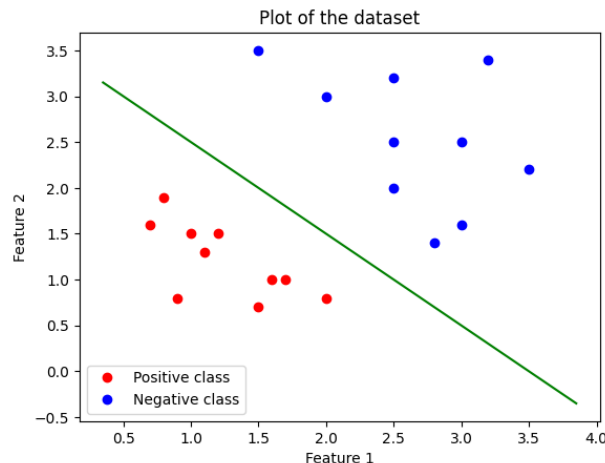
برای داده دوم، نقاط کلاس قرمز در دایره حول نقاط $(1, 1)$, $(3, 4)$ با شعاع یک قرار دارند. در نتیجه دو فیچر می توان اضافه کرد که فاصله نقاط را با این دو دایره داشته باشد، اگر فاصله نقطه ای با یکی از این دو نقطه از 1 کمتر باشد، این نقطه در کلاس قرمز قرار دارد.

2. پیاده سازی پرسپترون

ابتدا تابعی برای رسم نقاط می نویسیم و نقاط داده های تعریف شده را رسم می کنیم.

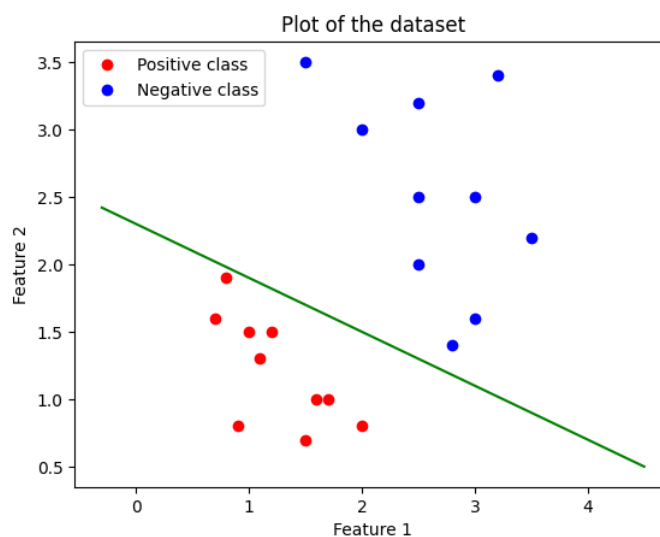


حال باید خطی پیدا کنیم که این نقاط را از هم جدا کند. این خط را بصورت زیر می توانیم رسم کنیم.



حال باید الگوریتم پرسپترون ساده را پیاده سازی و بر روی داده ها اعمال کنیم. الگوریتم پرسپترون ساده را با مقادیر متفاوت نرخ یادگیری، اجرا میکنیم. نتیجه بصورت زیر بدست آمده است.

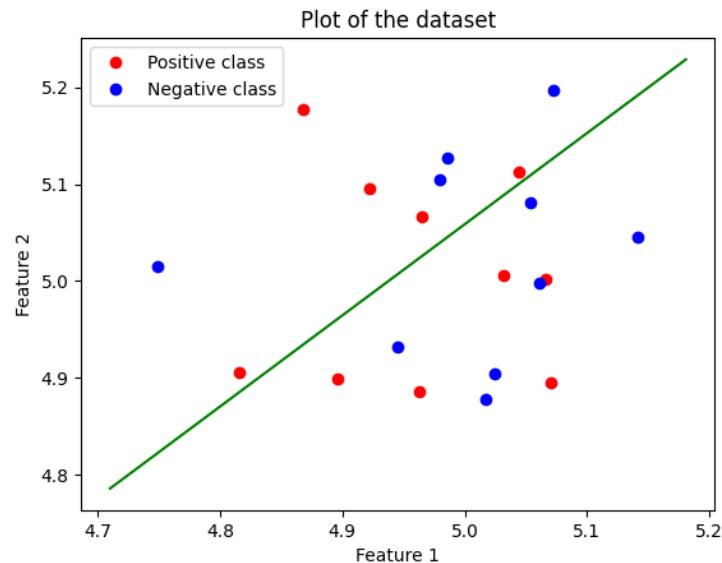
```
Learning rate: 1
Total updates: 46
Final update at iteration: 261
Optimal weights: [ -4. -10.]
Optimal bias: 23.0
-----
Learning rate: 0.1
Total updates: 46
Final update at iteration: 261
Optimal weights: [-0.4 -1. ]
Optimal bias: 2.3
-----
Learning rate: 0.01
Total updates: 46
Final update at iteration: 261
Optimal weights: [-0.04 -0.1 ]
Optimal bias: 0.22999999999999995
-----
Learning rate: 0.001
Total updates: 46
Final update at iteration: 261
Optimal weights: [-0.004 -0.01 ]
Optimal bias: 0.023000000000000007
-----
```



همانطور که مشاهده می شود، نتیجه با نرخ یادگیری های متفاوت هیچ تغییری نکرده است و فقط ضرایب در این نرخ یادگیری ضرب می شوند. مقادیر total update و final update at iteration به ترتیب تعداد کل بروزرسانی انجام شده بر روی خط جدا کننده و عدد آخرین بروزرسانی را نشان می دهد. از آنجاییکه تنها شرط جدا کنندگی برای پرسپترون در نظر گرفته شده است، اولین خطی که داده ها را جدا کند به عنوان خروجی داده شده است و این خط لزوماً بهترین خط با هر معیار نیست. به طور مثال خطی که بیشترین فاصله را با نقاط داشته باشد با این خط تفاوت دارد.

در قسمت بعدی پروژه، باید این الگوریتم را بر روی داده های رندم با مراکز و انحراف معیار مشخص، اجرا کنیم.

```
Learning rate: 0.1
Total updates: 26644
Final update at iteration: 150000
Optimal weights: [ 1.85058585 -1.96750827]
Optimal bias: 0.7
```



از آنجا که داده ها به صورت رندم تولید می شوند، لزوماً از یکدیگر بصورت خطی جدا پذیر نیستند در نتیجه ممکن خط جدا کنند پیدا نشود، مانند این مثال که خط بدست آمده داده ها را جدا نمی کند. در این مثال تعداد روزرسانی ها خیلی بالاست ولی ضرایب مقادیر پایینی دارند که می توان نتیجه گرفت این روزرسانی ها بصورت نوسانی انجام شده است، همچنین در آخرین پیمایش نیز روزرسانی داشته ایم، این یعنی نتیجه نهایی نباید نتیجه خوبی باشد که در نمودار مشخص است.

3. AutoEncoder

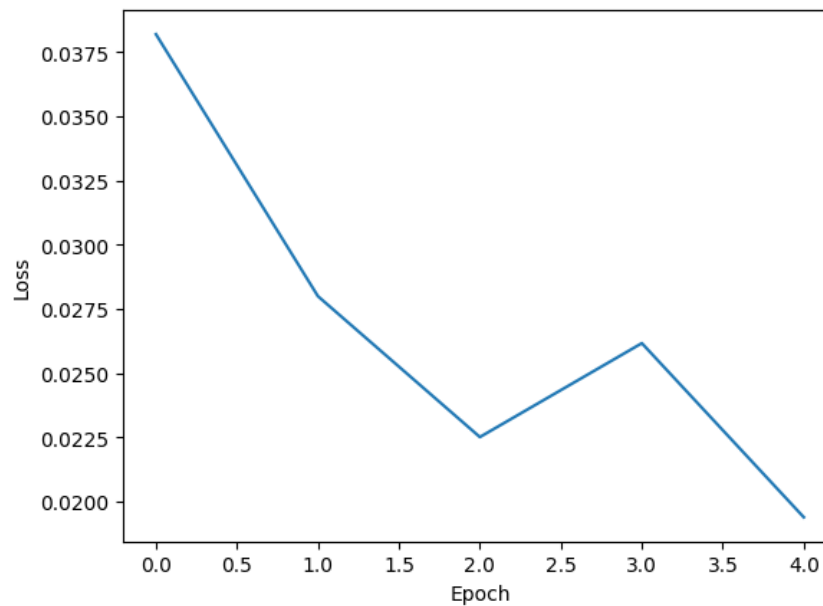
در این قسمت، هدف ساختن یک شبکه عصبی به عنوان AutoEncoder است. به این صورت که این شبکه داده های تصاویر ارقام دست نوشته را به فضای کوچک تر برده و سپس به فضای اصلی مدل کند. این شبکه از دو قسمت تشکیل می شود، قسمت اول بخش Encoder است که داده ها را به فضای کوچک تر می برد و قسمت دوم نیز Decoder این داده فضای کوچک تر را به فضای اصلی کد میکند. برای این مساله مدل زیر را تعریف کردیم:

```

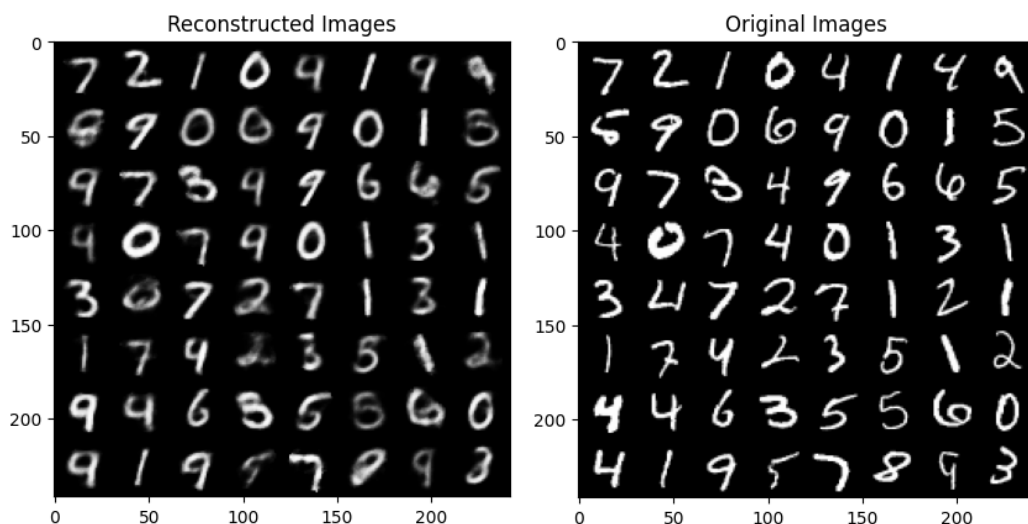
self.encoder = nn.Sequential(
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 16),
    nn.ReLU()
)
self.decoder = nn.Sequential(
    nn.Linear(16, 32),
    nn.ReLU(),
    nn.Linear(32, 64),
    nn.ReLU(),
    nn.Linear(64, 128),
    nn.ReLU(),
    nn.Linear(128, 28 * 28),
    nn.Sigmoid()
)

```

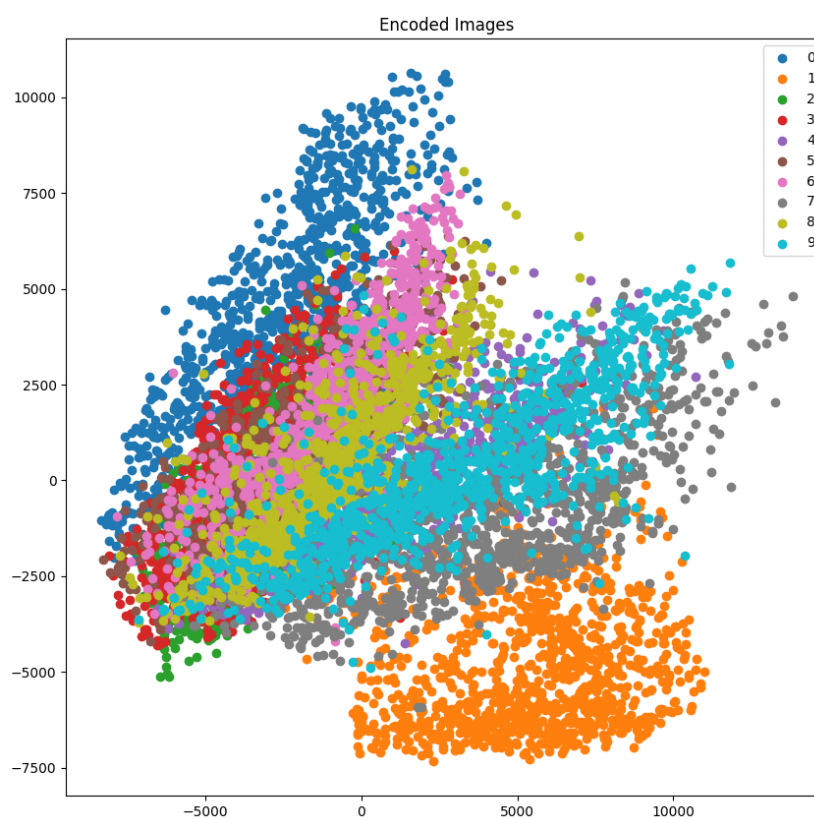
در این شبکه، فضای نهفته ابعاد 16 دارد. شبکه را بر روی داده های آموزشی Mnist با epoch 5 آموزش دادیم، نمودار loss به epoch بصورت زیر بدست آمد.



و تصاویر بازسازی شده در مقایسه با تصاویر اصلی برای داده های تست نیز بصورت زیر بدست آمد.

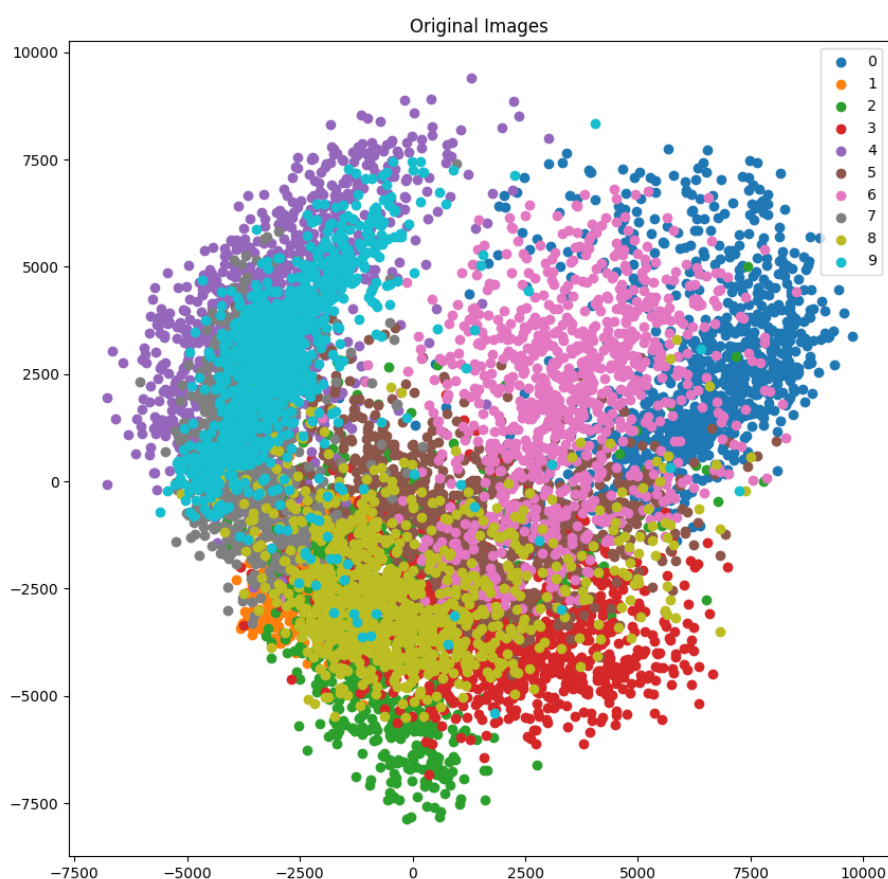


می توان دید که تصاویر بازسازی شده شباهت زیادی با تصاویر اصلی دارند که این یعنی به نظر مدل ما عملکرد خوبی داشته است. همچنین مقدار **loss** برای داده های تستی نیز 0.25 بدست آمد که مقدار نسبتاً خوبی است. حال داده های اصلی و داده های **encode** شده را با **isomap** به دو بعد برده و رسم میکنیم، نتیجه بصورت زیر بدست می آید:



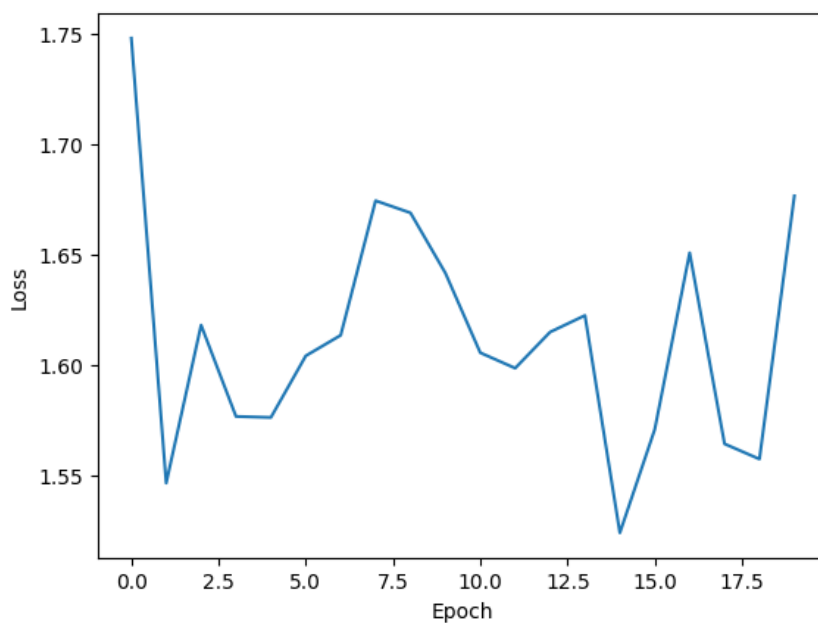
نقاط هم‌رنگ کنار یکدیگر قرار دارند، این یعنی داده‌های encode شده مربوط به هر لیبل در فضای نهفته نیز نزدیک یکدیگر هستند و در نتیجه می‌توان با اعمال الگوریتم‌های خوشه‌بندی روی داده‌های encode شده به جای اعمال آن به روی داده‌های اصلی با صرف هزینه محاسباتی کمتر نتایج مناسبی بدست بیاوریم.

البته اگر با دقت بیشتری به تصویر نگاه کنیم می‌بینیم داده‌های لیبل 4 تعداد می‌دارند و نزدیک داده‌های با لیبل 9 هستند این یعنی احتمالا تعداد زیادی از داده‌های لیبل 4 بصورت بد encode شده‌اند. برای اطمینان از میزان بد بودن این encode برای داده‌های این کلاس باید ماتریس درهم‌ریختگی و classification report را بررسی کنیم که در جلوتر این کار را انجام می‌دهیم، اما فعلا داده‌های اصلی را به دو بعد برده و رسم می‌کنیم.

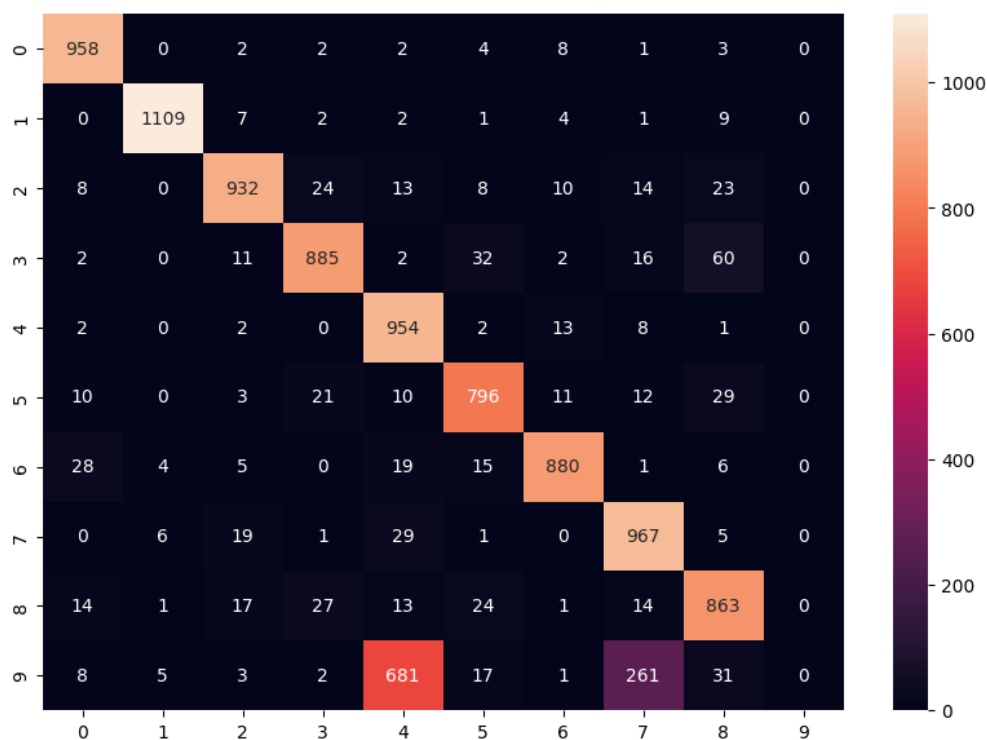


نکته‌ای که قابل اهمیت است شکل خوشه‌ها است، در داده‌های نهفته شکل همه خوشه‌ها تقریباً خطی است، ولی در داده‌های اصلی شکل خوشه‌ها متفاوت است و این باعث می‌شود داده‌های نهفته برای اعمال الگوریتم‌های خوشه‌بندی بهتر باشد. همچنین در نمودار داده‌های نهفته تفکیک بهتری انجام شده است.

در این قسمت از پروژه، یک شبکه عصبی دو لایه ای را بر روی داده های encode شده قبلی، آموزش می دهیم و نتایج را بررسی میکنیم. نمودار loss به epoch بصورت زیر بدست می آید.



نمودار بصورت نوسانی است و این یعنی احتمالا نیازی به این تعداد epoch نیست. ماتریس درهم‌ریختگی بصورت زیر بدست آمده است:



و classification report نیز بصورت زیر است:

Classification Report				
	precision	recall	f1-score	support
0	0.93	0.98	0.95	980
1	0.99	0.98	0.98	1135
2	0.93	0.90	0.92	1032
3	0.92	0.88	0.90	1010
4	0.55	0.97	0.70	982
5	0.88	0.89	0.89	892
6	0.95	0.92	0.93	958
7	0.75	0.94	0.83	1028
8	0.84	0.89	0.86	974
9	0.00	0.00	0.00	1009
accuracy			0.83	10000
macro avg	0.77	0.83	0.80	10000
weighted avg	0.77	0.83	0.80	10000

طبق این دو نمودار می توان دید که همه کلاس ها بجز کلاس 9 و 4 به خوبی توسط مدل آموزش دیده شده اند. مدل، کلاس داده های کلاس 9 را کلاس 4 پیش بینی کرده است و عملا هیچ داده کلاس 9 را به درستی پیش بینی نکرده است. البته می توان با بررسی تصاویر داده های بازسازی شده نیز این مورد را پیش بینی کرد، زیرا در این تصاویر داده های کلاس 9 به خوبی نهفته نشده اند و بسیار شبیه به 4 هستند. در نتیجه می توانستیم این موضوع را پیش بینی کنیم و لزوما اشکال از مدل نیست. با اینکه مدل همه داده های کلاس 9 را به اشتباه پیش بینی کرده است، باز نیز دقت کلی بسیار خوبی دارد. در نتیجه برای استفاده از این روش اگر بدانیم داده های کدام کلاس ها به شکل بدی نهفته می شوند و برای آن ها از روش دیگری استفاده کنیم آنگاه می توانیم انتظار دقت بالایی از نهفته کردن داده ها داشته باشیم.